

PHP - Variable Types

The main way to store information in the middle of a PHP program is by using a variable.

Here are the most important things to know about variables in PHP.

- ▣ All variables in PHP are denoted with a leading dollar sign (\$).
- ▣ The value of a variable is the value of its most recent assignment.
- ▣ Variables are assigned with the = operator, with the variable on the left-hand side and the expression to be evaluated on the right.
- ▣ Variables can, but do not need, to be declared before assignment.
- ▣ Variables in PHP do not have intrinsic types - a variable does not know in advance whether it will be used to store a number or a string of characters.
- ▣ Variables used before they are assigned have default values.
- ▣ PHP does a good job of automatically converting types from one to another when necessary.
- ▣ PHP variables are Perl-like.

PHP has a total of eight data types which we use to construct our variables –

- ▣ **Integers** – are whole numbers, without a decimal point, like 4195.
- ▣ **Doubles** – are floating-point numbers, like 3.14159 or 49.1.
- ▣ **Booleans** – have only two possible values either true or false.
- ▣ **NULL** – is a special type that only has one value: NULL.
- ▣ **Strings** – are sequences of characters, like 'PHP supports string operations.'
- ▣ **Arrays** – are named and indexed collections of other values.
- ▣ **Objects** – are instances of programmer-defined classes, which can package up both other kinds of values and functions that are specific to the class.
- ▣ **Resources** – are special variables that hold references to resources external to PHP (such as database connections).

The first five are *simple types*, and the next two (arrays and objects) are compound - the compound types can package up other arbitrary values of arbitrary type, whereas the simple types cannot.

We will explain only simple data type in this chapters. Array and Objects will be explained separately.

Integers

They are whole numbers, without a decimal point, like 4195. They are the simplest type .they correspond to simple whole numbers, both positive and negative. Integers can be assigned to variables, or they can be used in expressions, like so –

```
$int_var = 12345;  
$another_int = -12345 + 12345;
```

Integer can be in decimal (base 10), octal (base 8), and hexadecimal (base 16) format. Decimal format is the default, octal integers are specified with a leading 0, and hexadecimal have a leading 0x.

For most common platforms, the largest integer is $(2^{31} - 1)$ (or 2,147,483,647), and the smallest (most negative) integer is $-(2^{31} - 1)$ (or -2,147,483,647).

Doubles

They like 3.14159 or 49.1. By default, doubles print with the minimum number of decimal places needed. For example, the code –

[Live Demo](#)

```
<?php  
    $many = 2.2888800;  
    $many_2 = 2.2111200;  
    $few = $many + $many_2;  
  
    print("$many + $many_2 = $few <br>");  
?>
```

It produces the following browser output –

```
2.28888 + 2.21112 = 4.5
```

Boolean

They have only two possible values either true or false. PHP provides a couple of constants especially for use as Booleans: TRUE and FALSE, which can be used like so –

```
if (TRUE)  
    print("This will always print<br>");  
  
else  
    print("This will never print<br>");
```

Interpreting other types as Booleans

Here are the rules for determine the "truth" of any value not already of the Boolean type –

- If the value is a number, it is false if exactly equal to zero and true otherwise.
- If the value is a string, it is false if the string is empty (has zero characters) or is the string "0", and is true otherwise.
- Values of type NULL are always false.

- ☐ If the value is an array, it is false if it contains no other values, and it is true otherwise. For an object, containing a value means having a member variable that has been assigned a value.
- ☐ Valid resources are true (although some functions that return resources when they are successful will return FALSE when unsuccessful).
- ☐ Don't use double as Booleans.

Each of the following variables has the truth value embedded in its name when it is used in a Boolean context.

```
$true_num = 3 + 0.14159;  
$true_str = "Tried and true"  
$true_array[49] = "An array element";  
$false_array = array();  
$false_null = NULL;  
$false_num = 999 - 999;  
$false_str = "";
```

NULL

NULL is a special type that only has one value: NULL. To give a variable the NULL value, simply assign it like this –

```
$my_var = NULL;
```

The special constant NULL is capitalized by convention, but actually it is case insensitive; you could just as well have typed –

```
$my_var = null;
```

A variable that has been assigned NULL has the following properties –

- ☐ It evaluates to FALSE in a Boolean context.
- ☐ It returns FALSE when tested with IsSet() function.

Strings

They are sequences of characters, like "PHP supports string operations". Following are valid examples of string

```
$string_1 = "This is a string in double quotes";  
$string_2 = 'This is a somewhat longer, singly quoted string';  
$string_39 = "This string has thirty-nine characters";  
$string_0 = ""; // a string with zero characters
```

Singly quoted strings are treated almost literally, whereas doubly quoted strings replace variables with their values as well as specially interpreting certain character sequences.

```
<?php
    $variable = "name";
    $literally = 'My $variable will not print!';

    print($literally);
    print "<br>";

    $literally = "My $variable will print!";
    print($literally);
?>
```

This will produce following result –

```
My $variable will not print!
My name will print
```

There are no artificial limits on string length - within the bounds of available memory, you ought to be able to make arbitrarily long strings.

Strings that are delimited by double quotes (as in "this") are preprocessed in both the following two ways by PHP –

- Certain character sequences beginning with backslash (\) are replaced with special characters
- Variable names (starting with \$) are replaced with string representations of their values.

The escape-sequence replacements are –

- \n is replaced by the newline character
- \r is replaced by the carriage-return character
- \t is replaced by the tab character
- \\$ is replaced by the dollar sign itself (\$)
- \" is replaced by a single double-quote (")
- \\ is replaced by a single backslash (\)

Here Document

You can assign multiple lines to a single string variable using here document –

```
<?php
    $channel =<<<_XML_

    <channel>
        <title>What's For Dinner</title>
        <link>http://menu.example.com/ </link>
        <description>Choose what to eat tonight.</description>
    </channel>
    _XML_;
```

[Live Demo](#)

```
echo <<<END
This uses the "here document" syntax to output multiple lines with variable
interpolation. Note that the here document terminator must appear on a line with
just a semicolon. no extra whitespace!

END;

print $channel;

?>
```

This will produce following result –

```
This uses the "here document" syntax to output
multiple lines with variable interpolation. Note
that the here document terminator must appear on a
line with just a semicolon. no extra whitespace!

<channel>
<title>What's For Dinner</title>
<link>http://menu.example.com/</link>
<description>Choose what to eat tonight.</description>
```

Variable Scope

Scope can be defined as the range of availability a variable has to the program in which it is declared. PHP variables can be one of four scope types –

- ☐ Local variables 
- ☐ Function parameters 
- ☐ Global variables 
- ☐ Static variables 

Variable Naming

Rules for naming a variable is –

- ☐ Variable names must begin with a letter or underscore character.
- ☐ A variable name can consist of numbers, letters, underscores but you cannot use characters like + , - , % , (,) . & , etc

There is no size limit for variables.